

# Divide And Conquer Explanation

## 395. Longest Substring with At Least K Repeating Characters

We need to find the **longest substring** where:

Every character present in that substring appears **at least k times**.

For example:

```
s = "aaabb"
k = 3
```

- "aaa" → a appears 3 times ✓
- "aaabb" → a = 3, but b = 2 ✗

Answer = 3.

## The main idea behind Divide and Conquer

The most important observation is this:

Suppose we have a substring:

```
"aaabccddd"
k = 3
```

Frequency:

```
a → 3 ✓
b → 2 ✗
c → 2 ✗
d → 3 ✓
```

Now think about character **b**.

**b** appears only 2 times in the **entire current substring**.

But we need every character to appear at least 3 times.

Therefore:

**No valid substring can contain b.**

Why?

Because even if we choose a smaller substring containing **b**, its frequency can only stay the same or decrease.

```
Current substring: aaabccddd
```

```
b occurs = 2
```

```
Any smaller substring containing b:
b occurs ≤ 2
```

```
So it can never reach k = 3.
```

Therefore, **b** acts like a **wall**.

```
aaa | bb | cddd
    ↑
  invalid characters
```

We can split the problem around such invalid characters.

Then recursively solve the left and right parts.

This is the Divide and Conquer idea.

## Step-by-step intuition

Suppose:

```
s = "aaabb"
k = 3
```

### Step 1: Count frequencies

```
a → 3
b → 2
```

$b < k$ , so  $b$  cannot belong to any valid answer.

Split:

```
aaa | bb
```

Now solve each part separately:

```
solve("aaa")
solve("")
```

For "aaa":

```
a → 3 >= 3
```

Every character satisfies the condition.

So return:

```
3
```

## Recursive Divide and Conquer Logic

For every recursive substring:

### Case 1: Substring length is less than $k$

Example:

```
"ab"
k = 3
```

Even if both characters were the same, there aren't enough characters to have frequency 3.

So:

```
C++
if (right - left + 1 < k)
    return 0;
```

### Case 2: Count frequency of every character

Suppose:

```
substring = "aaabbcc"
k = 3

a = 3 ✓
b = 2 ✗
c = 2 ✗
```

Find a character whose frequency is  $< k$ .

That character cannot be part of the answer.

So divide around it.

### Case 3: No invalid character exists

Example:

```
"aaabbccccc"
k = 3
```

```
a = 3
b = 3
c = 3
```

Everything satisfies the condition.

Therefore the **entire substring is valid**.

Return its length immediately.

## C++ Divide and Conquer Solution

```
C++
class Solution {
public:
    int solve(string& s, int left, int right, int k) {
        // If substring length is smaller than k,
        // no character can appear at least k times.
        if (right - left + 1 < k) {
            return 0;
        }

        // Count frequency in current substring
        vector<int> freq(26, 0);

        for (int i = left; i <= right; i++) {
            freq[s[i] - 'a']++;
        }

        // Find a character whose frequency is < k
        for (int i = left; i <= right; i++) {
            if (freq[s[i] - 'a'] < k) {
                // This character cannot be part
                // of any valid substring.

                int next = i + 1;

                // Skip consecutive invalid characters
                while (next <= right &&
                    freq[s[next] - 'a'] < k) {
                    next++;
                }

                // Divide into two parts
                int leftPart = solve(s, left, i - 1, k);
                int rightPart = solve(s, next, right, k);

                return max(leftPart, rightPart);
            }
        }

        // Every character occurs at least k times
        return right - left + 1;
    }

    int longestSubstring(string s, int k) {
        return solve(s, 0, s.size() - 1, k);
    }
};
```

## Understanding the most important part

```
C++
if (freq[s[i] - 'a'] < k)
```

Suppose:

```
s = "aaabccddd"
k = 3
```

We find:

```
b → 2
```

So **b** is invalid.

We do:

```
C++
solve(left part)
solve(right part)
```

Conceptually:

```
aaabccddd
  ↑
  b is invalid

      Divide
    /      \
   aaa      ccddd
```

But there can be multiple consecutive invalid characters:

```
aaa | bbcc | ddd
```

Instead of recursively splitting one character at a time:

```
aaa | b | bccddd
```

then:

```
aaa | b | ccddd
```

we skip all consecutive invalid characters:

```
C++
int next = i + 1;
while (next <= right &&
      freq[s[next] - 'a'] < k) {
    next++;
}
```

This directly creates:

```
aaa | bbcc | ddd
```

Then:

```
C++
solve("aaa")
solve("ddd")
```

## Now let's do a proper hard dry run

Let's take:

```
s = "aaabbbccddeeeffffffggghhii"
k = 3
```

This is useful because it has multiple recursive divisions.

## Step 1: Initial call

```
C++
solve(s, 0, 26, 3)
```

String:

```
aaabbbccddeeeffffffggghhii
```

Frequency:

| Character | Frequency | Valid? |
|-----------|-----------|--------|
| a         | 3         | ✓      |
| b         | 3         | ✓      |
| c         | 2         | ✗      |
| d         | 2         | ✗      |
| e         | 4         | ✓      |
| f         | 5         | ✓      |
| g         | 3         | ✓      |
| h         | 3         | ✓      |
| i         | 2         | ✗      |

Invalid characters:

```
c, d, i
```

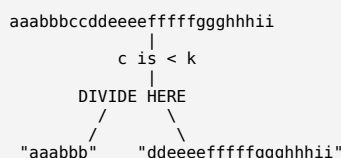
The first invalid character encountered is c.

## Recursion Tree – Level 1

```
solve("aaabbbccddeeeffffffggghhii")
```

Frequencies:

```
a=3 ✓
b=3 ✓
c=2 ✗
d=2 ✗
e=4 ✓
f=5 ✓
g=3 ✓
h=3 ✓
i=2 ✗
```



Because *c* and the next *c* and *d* are all invalid according to the frequency of the current substring, the algorithm skips consecutive invalid positions.  
So conceptually:

```
aaabbb | ccdd | eeeffffffggghhii
```

Thus:

```
C++
solve("aaabbb")
solve("eeeeffffffggghhii")
```

## Left recursive call

```
solve("aaabbb")
```

Frequency:

```
a = 3
b = 3
```

Both satisfy:

```
frequency >= k
```

Therefore:

```
C++
return 6;
```

Tree:

```

solve("aaabbb")
  |
  a = 3 ✓
  b = 3 ✓
  |
All characters valid
  |
  return 6
  
```

## Right recursive call

Now:

```
solve("eeeeffffffggghhii")
```

Frequency:

```
e = 4 ✓
f = 5 ✓
g = 3 ✓
h = 3 ✓
i = 2 ✗
```

*i* is invalid.

Split around *ii*:

```
eeeeffffffggghh | ii
```

Recursion:

```

solve("eeeeffffggghhii")
  |
  +-- i = 2 < 3
  |
  +-- divide around ii
  |
  +-- "eeeeffffggghh"
  |
  +-- ""

```

## Solve "eeeeffffggghh"

Frequency:

```

e = 4 ✓
f = 5 ✓
g = 3 ✓
h = 3 ✓

```

Everything is valid.

So:

```

C++
return 15;

```

Let's count:

```

eeee   = 4
fffff  = 5
ggg     = 3
hhh     = 3
-----
total  = 15

```

## Now recursion starts returning

The recursion tree:

```

solve("aaabbbccddeeeffffggghhii")
|-- invalid: c, d, i
+--- solve("aaabbb")
    |
    +--- all valid
    |
    +--- return 6
+--- solve("eeeeffffggghhii")
    |
    |-- i occurs 2 times < 3
    +--- solve("eeeeffffggghh")
        |
        +--- all valid
        |
        +--- return 15
    +--- solve("")
        |
        +--- length < k
        |
        +--- return 0
    max(15, 0)
    |
    return 15
max(6, 15)
|
return 15

```

Final answer:

```

15

```

The longest valid substring is:

```

eeeeffffggghh

```

## Let's understand WHY splitting works

This is the heart of the solution.

Suppose the current substring is:

```
aaabbbccddeee
```

and:

```
k = 3
```

Frequency:

```
a = 3 ✓
b = 3 ✓
c = 2 ✗
d = 2 ✗
e = 3 ✓
```

Consider any substring containing c.

For example:

```
"aaabbbc"
"ccdde"
"bbc"
"c"
```

The frequency of c can **never become greater** than its frequency in the original current substring.

Mathematically:

```
frequency(c in smaller substring)
<=
frequency(c in current substring)
```

Since:

```
frequency(c) = 2 < k
```

we know:

```
frequency(c in any smaller substring) < k
```

Therefore:

✗ No valid answer can cross through c.

So c becomes a divider:

```
[ possible answer ] | c | [ possible answer ]
```

That means we only need:

```
C++
max(
    solve(left),
    solve(right)
)
```

## A simpler version of the code

Here is another version that may be easier to understand:

```
C++
class Solution {
public:
    int helper(string& s, int start, int end, int k) {
        // Not enough characters
        if (end - start + 1 < k) {
            return 0;
        }

        vector<int> freq(26, 0);

        // Count characters
        for (int i = start; i <= end; i++) {
            freq[s[i] - 'a']++;
        }

        // Find invalid character
        for (int i = start; i <= end; i++) {
            if (freq[s[i] - 'a'] < k) {
                // Split from this position
                int leftAnswer =
                    helper(s, start, i - 1, k);
            }
        }
    }
};
```

```

        int rightAnswer =
            helper(s, i + 1, end, k);

        return max(leftAnswer, rightAnswer);
    }

    // No invalid character
    return end - start + 1;
}

int longestSubstring(string s, int k) {
    return helper(s, 0, s.size() - 1, k);
}
};
```

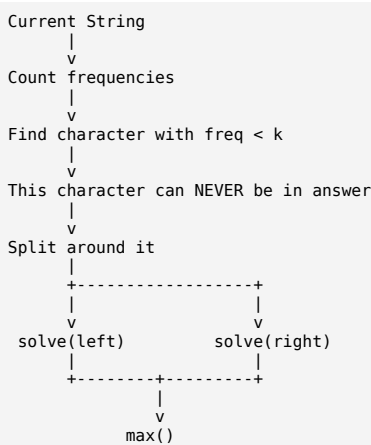
This works too, but the previous version that skips consecutive invalid characters is a little cleaner and can avoid unnecessary recursive calls.

## Divide and Conquer Pattern

The general pattern here is:

1. Solve the current problem.
2. Find something that makes the current range impossible.
3. That element can never belong to the answer.
4. Use it as a divider.
5. Recursively solve the independent parts.
6. Combine answers.

For this problem:



## Base cases

There are effectively two stopping situations.

### 1. Substring is too small

```
C++
if (right - left + 1 < k)
    return 0;
```

Example:

"ab"  
k = 3

Impossible.

Return:

0

**2. Every character has frequency  $\geq k$**

```
"aaabbbccc"  
k = 3
```



Everything is valid.

Return the entire length:

```
C++
return right - left + 1;
```

## Time Complexity

For every recursive call, we count frequencies:

```
O(n)
```

In the worst case, repeated recursive splitting can lead to:

```
O(n²)
```

However, because there are only **26 lowercase English characters**, the recursion depth/splitting behavior is practically bounded by the alphabet structure, and this divide-and-conquer approach is accepted efficiently for the given constraints.

## Space Complexity

```
O(26) + recursion stack
```

Ignoring the constant frequency array:

```
O(recursion depth)
```

For this problem, the effective recursion depth is bounded by the number of distinct lowercase characters, so auxiliary space is small.

## The one-line intuition to remember

**If a character occurs fewer than k times in the current substring, no valid answer can contain that character—so split the string around it and recursively find the best answer on both sides.**

ChatGPT can make mistakes. Check important info.